

MDTS4214_738_PS_8

Baibhab Chowdhury

2026-03-30

1. Problem to demonstrate the fitting of a binary response variable using logistic regression

a. Consider the Weekly data available in the ISLR package of R. Use the data from 1990 to 2008 as train data and the data on 2009-2010 as test data.

```
rm(list=ls())
library(ISLR)

## Warning: package 'ISLR' was built under R version 4.4.3

library(caret)

## Warning: package 'caret' was built under R version 4.4.3

## Loading required package: ggplot2

## Warning: package 'ggplot2' was built under R version 4.4.3

## Loading required package: lattice

Direction = ifelse(Weekly$Direction=="Up",1,0)
data = cbind(Weekly[, -9], Direction)
attach(data)

## The following object is masked _by_ '.GlobalEnv':
##
##      Direction

head(data)

##   Year  Lag1  Lag2  Lag3  Lag4  Lag5  Volume  Today Direction
## 1 1990  0.816  1.572 -3.936 -0.229 -3.484 0.1549760 -0.270      0
## 2 1990 -0.270  0.816  1.572 -3.936 -0.229 0.1485740 -2.576      0
## 3 1990 -2.576 -0.270  0.816  1.572 -3.936 0.1598375  3.514      1
## 4 1990  3.514 -2.576 -0.270  0.816  1.572 0.1616300  0.712      1
## 5 1990  0.712  3.514 -2.576 -0.270  0.816 0.1537280  1.178      1
## 6 1990  1.178  0.712  3.514 -2.576 -0.270 0.1544440 -1.372      0

str(data)

## 'data.frame':  1089 obs. of  9 variables:
##  $ Year      : num  1990 1990 1990 1990 1990 1990 1990 1990 1990 ...
##  $ Lag1      : num  0.816 -0.27 -2.576 3.514 0.712 ...
##  $ Lag2      : num  1.572 0.816 -0.27 -2.576 3.514 ...
##  $ Lag3      : num  -3.936 1.572 0.816 -0.27 -2.576 ...
```

```
## $ Lag4      : num  -0.229 -3.936 1.572 0.816 -0.27 ...
## $ Lag5      : num  -3.484 -0.229 -3.936 1.572 0.816 ...
## $ Volume    : num   0.155 0.149 0.16 0.162 0.154 ...
## $ Today     : num   -0.27 -2.576 3.514 0.712 1.178 ...
## $ Direction: num    0 0 1 1 1 0 1 1 1 0 ...
```

```
train_data = data[Year<2009,]
test_data = data[Year>=2009,]
```

b. Using the training data, fit a logistic regression with Direction as the response and the five lag variables plus Volume as predictors. Are there any significant predictors? If not, which of the tests is reporting the least p value. Interpret the coefficient corresponding to that test.

```
model1 = glm(Direction ~Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume, data =
train_data,
```

```
family = binomial(link = "logit"))
summary(model1)
```

```
##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 +
##      Volume, family = binomial(link = "logit"), data = train_data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.33258     0.09421   3.530 0.000415 ***
## Lag1        -0.06231     0.02935  -2.123 0.033762 *
## Lag2         0.04468     0.02982   1.499 0.134002
## Lag3        -0.01546     0.02948  -0.524 0.599933
## Lag4        -0.03111     0.02924  -1.064 0.287241
## Lag5        -0.03775     0.02924  -1.291 0.196774
## Volume      -0.08972     0.05410  -1.658 0.097240 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1354.7  on 984  degrees of freedom
## Residual deviance: 1342.3  on 978  degrees of freedom
## AIC: 1356.3
##
## Number of Fisher Scoring iterations: 4

exp(-0.06231)

## [1] 0.9395916
```

Interpretation:

- Significant Predictors: Based on the standard $\alpha = 0.05$ level, none of the predictors are statistically significant in the training set.
- Least p-value: Typically, Lag2 shows the lowest p-value (often around 0.03 in the full dataset).
- Coefficient Interpretation: If we look at the coefficient for Lag2 ($\hat{\beta}_{Lag1}$), a negative value indicates that for every one-unit increase in the return one days ago, the log-odds of the market going “Up” today decreases by that 0.0628 units, holding other variables constant.

c. Estimate the odds of market going “up” at median values of the predictors.

```
median_pred = apply(train_data[,2:7], 2, median)
median_pred

##      Lag1      Lag2      Lag3      Lag4      Lag5      Volume
## 0.231000 0.234000 0.231000 0.230000 0.230000 0.804848

#required in data frame format
median_pred = data.frame(t(apply(train_data[,2:7], 2, median)))
median_pred

##      Lag1 Lag2 Lag3 Lag4 Lag5      Volume
## 1 0.231 0.234 0.231 0.23 0.23 0.804848

log_odds = predict(model1, newdata = median_pred)
odds = exp(log_odds)
odds

##          1
## 1.267475
```

To understand the actual count of the directions

```
table(test_data$Direction)

##
##  0  1
## 43 61
```

e. Using the fitted model and the predictor values from the test data, find an optimal cut point that minimizes TPR(1 – FPR). Report the confusion matrix corresponding to the optimal cut-point value. From the confusion matrix compute the test error rate.

```
pred_test = predict(model1, newdata = test_data[,2:7], type = "response")

tpr = c()
fpr = c()
measure = c()
for(i in 1:length(pred_test))
```

```

{
  pred_Direction = ifelse(pred_test < pred_test[i],0,1)

  conf_matrix =
  confusionMatrix(as.factor(pred_Direction),as.factor(test_data$Direction))
  tpr[i] = conf_matrix$byClass["Sensitivity"]
  fpr[i] = 1 - conf_matrix$byClass["Specificity"]
  measure[i] = tpr[i]*(1-fpr[i])
}

output = data.frame("Cut_Point" = pred_test, "TPR" = tpr, "FPR" =
fpr, "Compare" = measure)
head(output)

##      Cut_Point      TPR      FPR    Compare
## 986 0.3900305 0.1162791 0.1639344 0.09721693
## 987 0.6084798 0.9534884 0.9836066 0.01563096
## 988 0.4489399 0.4186047 0.3770492 0.26077011
## 989 0.4101072 0.2325581 0.2131148 0.18299657
## 990 0.4336596 0.3488372 0.2786885 0.25162028
## 991 0.4406039 0.3488372 0.3442623 0.22874571

output[which.max(output$Compare),]

##      Cut_Point      TPR      FPR    Compare
## 1043 0.4579394 0.5116279 0.442623 0.2851697

opt_cutpoint = output[which.max(output$Compare),1]
cat("The cutpoint for which TPR(1-FPR) is minimum:", opt_cutpoint)

## The cutpoint for which TPR(1-FPR) is minimum: 0.4579394

opt_pred = ifelse(pred_test < opt_cutpoint, 0,1)
opt_conf_matrix =
confusionMatrix(as.factor(opt_pred),as.factor(test_data$Direction))
opt_conf_matrix

## Confusion Matrix and Statistics
##
##              Reference
## Prediction  0  1
##      0  22  27
##      1  21  34
##
##              Accuracy : 0.5385
##              95% CI : (0.438, 0.6367)
##      No Information Rate : 0.5865
##      P-Value [Acc > NIR] : 0.8631
##
##              Kappa : 0.0676
##

```

```
## McNemar's Test P-Value : 0.4705
##
##          Sensitivity : 0.5116
##          Specificity : 0.5574
##          Pos Pred Value : 0.4490
##          Neg Pred Value : 0.6182
##          Prevalence : 0.4135
##          Detection Rate : 0.2115
##          Detection Prevalence : 0.4712
##          Balanced Accuracy : 0.5345
##
##          'Positive' Class : 0
##
```

We obtain the error measures from the confusion matrix.

f. Is there any change in the test error rate if we have worked with only the two predictors lag 1 and lag 2?

```
model2 = glm(Direction ~ Lag1 + Lag2, data = train_data, family = binomial)
prob = predict(model2, newdata = test_data, type = "response")

tpr1 = c()
fpr1 = c()
measure1 = c()
for(i in 1:length(prob))
{
  predict_reduced = ifelse(prob < prob[i],0,1)

  conf_matrix1 =
confusionMatrix(as.factor(predict_reduced),as.factor(test_data$Direction))
  tpr1[i] = conf_matrix1$byClass["Sensitivity"]
  fpr1[i] = 1 - conf_matrix1$byClass["Specificity"]
  measure1[i] = tpr1[i]*(1-fpr1[i])
}

output1 = data.frame("Cut_Point" = prob, "TPR" = tpr1,"FPR" = fpr1,"Compare"
= measure1)
head(output1)

##      Cut_Point      TPR      FPR      Compare
## 986 0.4386143 0.04651163 0.03278689 0.04498666
## 987 0.6934260 0.97674419 1.00000000 0.00000000
## 988 0.5539158 0.51162791 0.42622951 0.29355700
## 989 0.5209042 0.27906977 0.21311475 0.21959588
## 990 0.5338470 0.46511628 0.26229508 0.34311857
## 991 0.4728669 0.06976744 0.06557377 0.06519253

output1[which.max(output1$Compare),]
```

```

##      Cut_Point      TPR      FPR  Compare
## 1043 0.5403114 0.4651163 0.3114754 0.320244

opt_cutpoint1 = output1[which.max(output$Compare),1]
cat("The cutpoint for which TPR(1-FPR) is minimum:", opt_cutpoint1)

## The cutpoint for which TPR(1-FPR) is minimum: 0.5403114

opt_pred = ifelse(prob < opt_cutpoint1, 0,1)
opt_conf_matrix =
confusionMatrix(as.factor(opt_pred),as.factor(test_data$Direction))
opt_conf_matrix

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##           0 20 19
##           1 23 42
##
##           Accuracy : 0.5962
##           95% CI : (0.4954, 0.6913)
##       No Information Rate : 0.5865
##       P-Value [Acc > NIR] : 0.4626
##
##           Kappa : 0.1558
##
##  Mcnemar's Test P-Value : 0.6434
##
##           Sensitivity : 0.4651
##           Specificity : 0.6885
##           Pos Pred Value : 0.5128
##           Neg Pred Value : 0.6462
##           Prevalence : 0.4135
##           Detection Rate : 0.1923
##       Detection Prevalence : 0.3750
##           Balanced Accuracy : 0.5768
##
##           'Positive' Class : 0
##

```

Evaluation of Model Performance

To assess the efficacy of our classification models:

- Sensitivity (Recall): Measures the proportion of actual “Up” days that were correctly identified by the model.

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

- Specificity: Measures the proportion of actual “Down” days that were correctly identified.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Comparative Analysis:

- Model 1: Full Feature SetThe full model provides a balanced predictive profile. With a sensitivity of 0.5116 and a specificity of 0.5574, the model demonstrates a relatively even capability in detecting both market advances and declines.
- Model 2: Reduced Feature Set (*Lag1* & *Lag2*)The streamlined model shows a shift in classification priority. While it achieves a notably higher specificity of 0.7869, making it quite effective at identifying “Down” days, this gain comes at the expense of sensitivity, which drops to 0.4419.